

How We Fetch “Raw” Satellite Indices

Technical guide for the AdvaRisk / TerraDrishti pipeline — NDVI, EVI, RVI, report charts, and monsoon data points

Repository: AdvaRisk Test

Primary files: `data_loader.py`, `analytics_engine.py`, `api.py`,
`templates/report.html`

Data source: Google Earth Engine (GEE)

Generated: 25 May 2026, 05:41 PM

1. What this document explains

Intelligence reports show a **TREND ANALYSIS (NDVI/EVI/RVI)** chart with two visual layers: **smoothed lines** and **scatter markers labeled “(raw)”**. This guide explains, step by step, how those values are produced in code, what “raw” really means in this project, and why you can still see optical (NDVI/EVI) points during the Indian monsoon (June–July and beyond).

Key takeaway: “Raw” in our reports means *pre-Whittaker smoothing in Python* — not unprocessed satellite bands and not necessarily cloud-free parcel observations.

2. End-to-end pipeline (fetch → chart → PDF)

```
API /test/accuracy
→ satellite_worker()
  → run_full_analytics_pipeline()    [analytics_engine.py]
    → process_parcel_data()          [data_loader.py – GEE fetch]
    → raw_indices_df snapshot        [before smoothing]
    → dataset_df                     [daily grid + Whittaker smooth]
    → Plotly chart (lines + markers)
    → ndvi_b64 (PNG base64)
  → generate_intelligence_report()    [pdf_generator.py]
    → report.html embeds satellite.images.ndvi_b64
```

Entry point in the API:

`api.py`

```
def satellite_worker(task_id, coords, end_date):
    return run_full_analytics_pipeline(task_id, coords, end_date)
```

3. Google Earth Engine setup

Before any imagery is requested, Earth Engine is initialized with project credentials (service account in production, or local `gcloud` auth).

`data_loader.py`

```
def initialize_ee():
    service_account = os.getenv("GEE_SERVICE_ACCOUNT")
    ...
    ee.Initialize(credentials, project=project_id) # or local default

initialize_ee() # runs at import; called again inside process_parcel_data()
```

4. Parcel geometry and date window

The parcel is a polygon from KML coordinates. The analysis window is **one calendar year** ending on the user-supplied `end_date`.

`data_loader.py` — `process_parcel_data()`

```
end_date = pd.to_datetime(end_str)
start_date = end_date - pd.DateOffset(years=1)
start_str = start_date.strftime('%Y-%m-%d')

polygon = ee.Geometry.Polygon([coordinates])
```

5. Sentinel-2: cloud mask and index formulas (NDVI, EVI)

We use the harmonized surface reflectance collection `COPERNICUS/S2_SR_HARMONIZED`. Cloudy pixels are masked using the Scene Classification Layer (SCL), then reflectance is scaled to 0–1.

5.1 Cloud masking (pixel level)

SCL classes removed: **3** (cloud shadows), **8** (medium cloud), **9** (high cloud), **10** (thin cirrus).

```
def maskS2clouds(image):
    scl = image.select('SCL')
    mask = (
        scl.neq(3).And(scl.neq(8))
        .And(scl.neq(9)).And(scl.neq(10))
    )
    return image.updateMask(mask).divide(10000).copyProperties(image,
["system:time_start"])
```

5.2 NDVI and EVI (computed on each image in GEE)

Index	Bands	Formula in code
NDVI	B8 (NIR), B4 (RED)	<code>normalizedDifference(['B8', 'B4'])</code> → $(\text{NIR} - \text{RED}) / (\text{NIR} + \text{RED})$
EVI	B8, B4, B2 (BLUE)	<code>2.5 * (NIR-RED) / (NIR + 6*RED - 7.5*BLUE + 1)</code>

```
def compute_indices(img):
    ndvi = img.normalizedDifference(['B8', 'B4']).rename('NDVI')
    evi = img.expression(
        '(2.5*((NIR-RED)/(NIR+6*RED-7.5*BLUE+1)))',
        {'NIR': img.select('B8'), 'RED': img.select('B4'), 'BLUE': img.select('B2')}
    ).rename('EVI')
    return img.addBands([ndvi, evi]).select(['NDVI', 'EVI'])
```

5.3 Scene-level cloud filter + full-scene fetch

- **Scene filter:** entire tile must have `CLOUDY_PIXEL_PERCENTAGE < 20` (metadata for the granule, not your parcel alone).
- **Current behavior:** all Sentinel-2 scenes that pass the filters are fetched. The old `limit_to_60()` thinning call is intentionally commented out.

```
s2_col = (ee.ImageCollection('COPERNICUS/S2_SR_HARMONIZED')
    .filterDate(start_str, end_str)
    .filterBounds(polygon)
    .filter(ee.Filter.lt('CLOUDY_PIXEL_PERCENTAGE', 20))
    .map(maskS2clouds)
    .map(compute_indices))
# s2_col = limit_to_60(s2_col)
```

6. Sentinel-1: RVI (radar — works through clouds)

Radar is not blocked by monsoon clouds. RVI is computed from VV and VH in decibels, converted to linear power first.

```
def compute_rvi(img):
    vv = ee.Image(10).pow(img.select('VV').divide(10))
    vh = ee.Image(10).pow(img.select('VH').divide(10))
    rvi = vh.multiply(4).divide(vv.add(vh)).rename('RVI')
    return img.addBands(rvi).select(['RVI'])

s1_col = (ee.ImageCollection('COPERNICUS/S1_GRD')
          .filterDate(start_str, end_str)
          .filterBounds(polygon)
          .filter(ee.Filter.eq('instrumentMode', 'IW'))
          .filter(ee.Filter.listContains('transmitterReceiverPolarisation', 'VV'))
          .filter(ee.Filter.listContains('transmitterReceiverPolarisation', 'VH'))
          .filter(ee.Filter.eq('orbitProperties_pass', 'DESCENDING'))
          .map(compute_rvi))
```

Note: Sentinel-1 was already not passed through `limit_to_60`. Sentinel-2 and Dynamic World are now also fetched without that thinning step.

7. What `extract_timeseries` means

This is a **local helper function** (not a GEE API name). It converts an `ImageCollection` into a **pandas table**: one row per satellite acquisition date, one column per index/band.

For **each image** in the collection it:

1. Computes the **mean** of each band inside the parcel polygon at **10 m** scale.
2. Stores the image acquisition date as `YYYY-MM-dd`.
3. Downloads all rows from GEE with `getInfo()` and builds a sorted DataFrame.

```
def extract_timeseries(collection, bands):
    def reducer(img):
        stats = img.reduceRegion(
            reducer=ee.Reducer.mean(),
            geometry=polygon,
            scale=10,
            bestEffort=True,
            maxPixels=1e13
        )
        props = {'date': img.date().format('YYYY-MM-dd')}
        for b in bands:
            props[b] = stats.get(b)
        return ee.Feature(None, props)
    fc = collection.map(reducer).filter(ee.Filter.notNull(bands))
    info = fc.getInfo().get('features', [])
    rows = [f['properties'] for f in info]
    df = pd.DataFrame(rows)
    df['date'] = pd.to_datetime(df['date'])
    return df.sort_values('date')

df_s2 = extract_timeseries(s2_col, ['NDVI', 'EVI'])
df_s1 = extract_timeseries(s1_col, ['RVI'])
```

Important: `bestEffort=True` allows a mean even when only part of the parcel has valid pixels after cloud masking. There is **no rule** requiring e.g. 50% of the parcel to be cloud-free. This matters heavily during monsoon.

8. Merging sensors into `df_all`

S2, S1, and Dynamic World are merged on `date` with an **outer join** — dates from each sensor are kept even when the others are missing (NaN).

```
df_all = df_s2.copy()
df_all = df_all.merge(df_s1, on='date', how='outer')
df_all = df_all.merge(df_dw, on='date', how='outer')
df_all = df_all.sort_values('date')
```

Result: on an S2 date you may have NDVI+EVI but NaN for RVI; on an S1 date, RVI only.

9. What “raw” scatter points are in the report chart

Immediately after GEE extraction, we snapshot index columns **before** Whittaker smoothing or daily resampling.

`analytics_engine.py`

```

_raw_index_cols = [c for c in ['NDVI', 'EVI', 'RVI'] if c in df_all.columns]
raw_indices_df = (
    df_all[['date'] + _raw_index_cols]
    .copy()
    .assign(date=lambda d: pd.to_datetime(d['date']).dt.normalize())
    .dropna(subset=_raw_index_cols, how='all')
    .drop_duplicates(subset=['date'])
    .sort_values('date')
)

```

9.1 How the chart is drawn

Visual	Data source	Meaning
Lines — “(smoothed)”	<code>dataset_df</code> after daily resample + Whittaker	Continuous ~365-day series used for predictions
Markers — “(raw)”	<code>raw_indices_df</code>	One point per acquisition date where that index is non-NaN

```

# Smoothed lines
for col in ['NDVI', 'EVI', 'RVI']:
    fig_indices.add_trace(go.Scatter(
        x=test_df['date'], y=test_df[col], mode='lines', name=f'{col} (smoothed)'))

# Raw markers — per column, drop NaN for that index only
raw_col = raw_indices_df.dropna(subset=[col])
fig_indices.add_trace(go.Scatter(
    x=raw_col['date'], y=raw_col[col], mode='markers', name=f'{col} (raw)'))

```

The PNG is embedded in the PDF as `satellite.images.ndvi_b64` in `templates/report.html`.

10. What happens after “raw” — smoothing (not shown as scatter)

```

dataset_df = df_all_clean.merge(df_dw_raw, on='date', how='outer')
dataset_df = dataset_df.groupby('date').mean()
dataset_df = dataset_df.resample('D').mean() # daily grid

dataset_df['NDVI'] = whittaker_smooth(..., lambda_=50)
dataset_df['EVI'] = whittaker_smooth(..., lambda_=50)
dataset_df['RVI'] = whittaker_smooth(..., lambda_=200)

```

The **annexure table** in the PDF uses `crop_activity_predictions_list`, which comes from this **smoothed daily** dataframe — not from `raw_indices_df`. Table values and scatter markers will not match on the same dates.

11. Why monsoon (June–July) still shows NDVI/EVI points

Over much of India, the southwest monsoon (roughly **June–September**) reduces usable optical imagery. Many users expect *no* NDVI points in July. Our pipeline can still show some — for technical reasons below. This is usually **sparse real observations**, not invented dates, but **confidence is lower**.

11.1 Scene cloud % is for the whole tile, not your field

`CLOUDY_PIXEL_PERCENTAGE < 20` applies to the entire Sentinel-2 granule. A scene can pass while your parcel still has patchy clouds — or a relatively clear field can contribute to a scene that passes the 20% rule.

11.2 SCL masking is imperfect + partial means

After SCL removes cloud classes, remaining pixels are averaged. With `bestEffort=True`, GEE may return a mean from only the clear fraction of the parcel. Monsoon points often mean: *“we got some masked NDVI,”* not *“full clear-sky view.”*

11.3 Monsoon ≠ zero satellite passes

Sentinel-2 revisits every few days. Even in cloudy months, a fraction of scenes pass filters. Because the current code no longer applies `limit_to_60`, all filtered June/July optical scenes are eligible to appear as raw markers.

11.4 Do not confuse NDVI with RVI in monsoon

RVI (orange markers) comes from Sentinel-1 radar and is largely unaffected by clouds. **NDVI/EVI (green/blue markers)** are optical. Monsoon charts often look “fuller” because of SAR — check the legend.

11.5 Smoothed lines can hide data gaps

A continuous green line through July does **not** mean NDVI was measured every day. Whittaker smoothing bridges gaps between sparse scene dates. Always compare **marker density** to the line.

12. Does this create false data or reduce accuracy?

Question	Answer
Are scatter dates fabricated?	No — each marker ties to a real filtered GEE scene date.

Are daily smoothed values measured?	Not all — resample + Whittaker synthesize values on days without observations.
Are monsoon optical points always wrong?	No — but they are often noisier and should be lower confidence.
Impact on crop verdict?	Predictions use smoothed daily series — monsoon gaps can be under-filled or over-smoothed , affecting activity %.

13. Glossary

Term	In this project
GEE	Google Earth Engine — cloud platform for satellite processing
extract_timeseries	Helper: ImageCollection → pandas (parcel mean per date)
raw_indices_df	Pre-Whittaker snapshot for chart markers
dataset_df	Daily smoothed series for ML logic and annexure
Outer join	Keep all dates from S2, S1, DW even when other sensors missing
Whittaker	Smoothing algorithm to denoise and fill gaps on daily grid

14. Possible improvements (future work)

- Require minimum % of parcel pixels valid after cloud mask before keeping a scene.
- Tighten `CLOUDY_PIXEL_PERCENTAGE` (e.g. < 10) or add parcel-level QA.
- Plot optical and radar on separate panels; annotate scene count per month on the PDF.
- Store true scene-level series in pickle (`vegetation_indices` currently saves smoothed daily data).
- Flag annexure rows as *observed vs interpolated/smoothed*.